

Logical Operations

➤ This operators are used for masking. White pixels are represented by binary 1 while black pixels are represented by binary 0.

➤ AND

For any two digital images, say f and g , the AND would yield the output image z with the logical AND of the pixel values of f and g at the (x, y) co-ordinates respectively.

➤ NOT

impact of NOT is exactly same as achieving the Negative Image

➤ OR

For any two digital images, say f and g , the OR would yield the output image z with the logical OR of the pixel values of f and g at the (x, y) co-ordinates respectively.

AND Operation

IMAGE 1

0	0	0	255
0	0	0	255
0	0	0	255
255	255	255	255

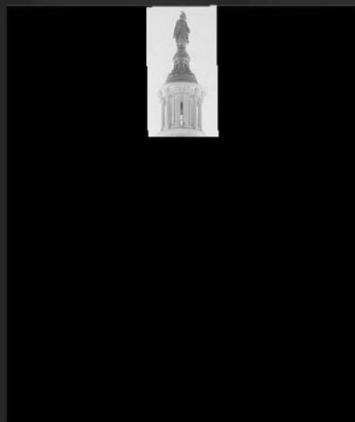
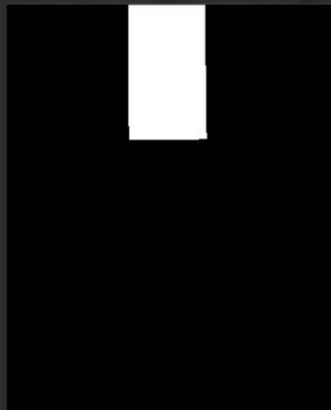
IMAGE 2

255	255	255	255
255	0	0	0
255	0	0	0
255	0	0	0

1 AND 2

0	0	0	255
0	0	0	0
0	0	0	0
255	0	0	0

Logical Operation (AND)



```
clear;  
clc;  
img3=imread('building.jpg');  
img3=rgb2gray(img3);  
BW=roipoly(img3);  
%%  
img3=double(img3);  
BW=double (BW)*255;  
final=bitand(img3,BW);  
img3=mat2gray(img3);  
BW=mat2gray(BW);  
final=mat2gray(final);  
figure, imshow(img3);  
figure, imshow(BW);  
figure, imshow (final);
```

OR Operation

IMAGE 1

0	0	0	255
0	0	0	255
0	0	0	255
255	255	255	255

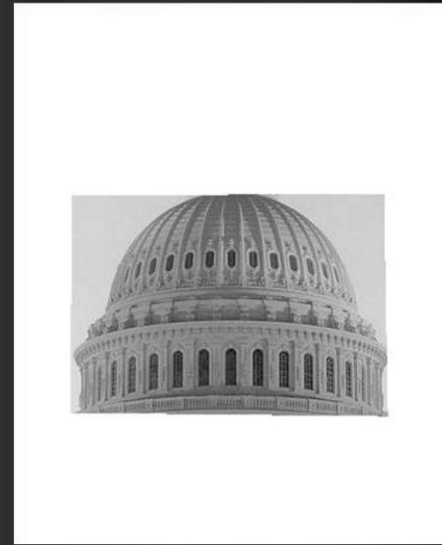
IMAGE 2

255	255	255	255
255	0	0	0
255	0	0	0
255	0	0	0

1 OR 2

255	255	255	255
255	0	0	255
255	0	0	255
255	255	255	255

Logical Operation (OR)



Logical Operation (OR)

```
clear;  
clc;  
img3=imread('building.jpg');  
img3=rgb2gray(img3);  
BW=roipoly(img3);  
BW=imcomplement(BW);  
BW=double(BW)*255;  
img3=double(img3);  
final=bitor(img3, BW);  
img3=mat2gray(img3);  
BW=mat2gray(BW);  
final=mat2gray(final);  
figure, imshow(img3);  
figure, imshow(BW);  
figure, imshow (final);
```

Image Transformation

- Geometric transformation modify spatial relationship between the pixels in an image. These are known as rubber-sheet Transformation.
- Transformation includes basically three operations : Translation, scaling and rotation
- Scaling indicates zooming of an image or shrinking of an image
- Translation translates an image into another form.

Example is

$$A = A + 100$$

Image Transformation (Rotation)

```
a=imread('D:\livingroom_gray.bmp');  
[m, n]=size (a)  
b=zeros(m,n);  
for i=1:1:m  
    b(:, i)=a(i, :);  
end  
imshow(mat2gray(b))
```


Image Transformation (Rotation)

- Rotation rotates image in a counter clockwise direction

Function to rotate an image is

`rot90(image matrix, k*90)`

Take $k=1$;

`rot90(image matrix, 90)`

Take $k = 2$;

`rot90(image matrix, 180)`

- Limitation of this function is it cannot take values of k like 1.5. 0.5 etc.

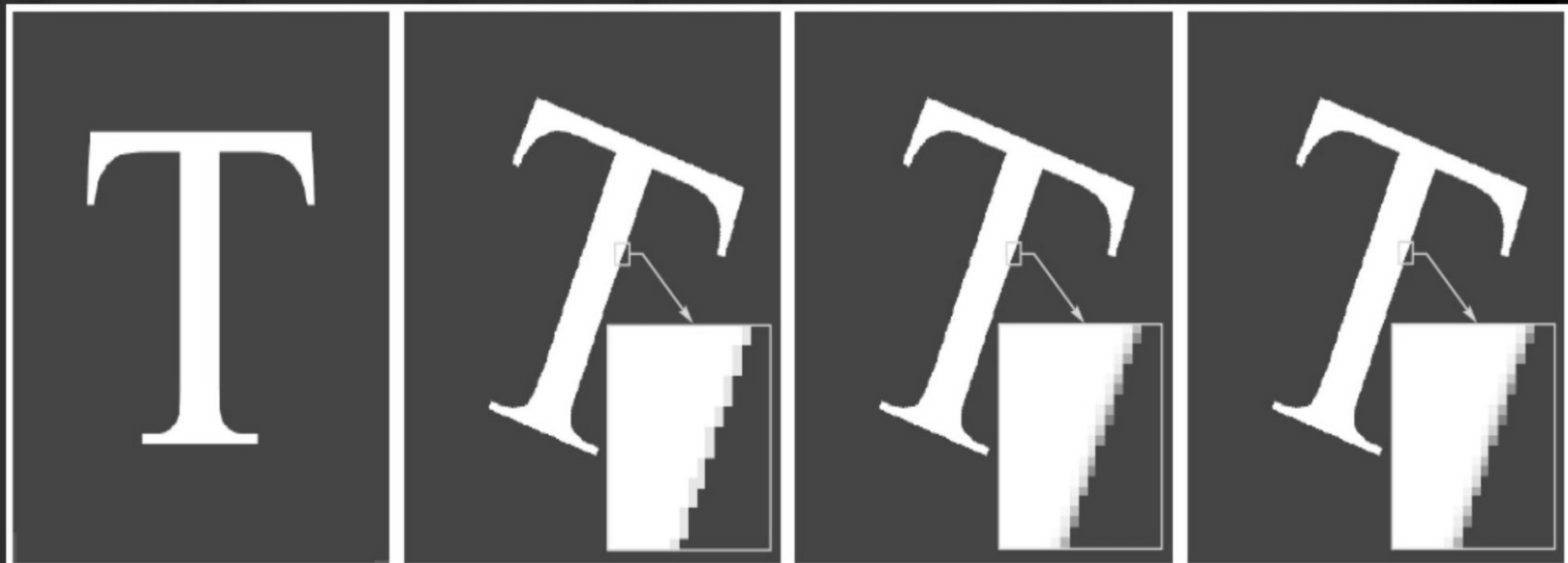
Image Transformation (Rotation)

- imrotate function rotates image in a counter clockwise direction

$B = \text{imrotate}(\text{image matrix}, \text{angle}, \text{method}, \text{bounding box})$

- Here, rotation angle can be any value
- Method indicates interpolation method like , nearest neighbor, bilinear etc.
- When Bounding box is 'loose', B includes the whole rotated image, which generally is larger than A. When Bounding box is 'crop' B is cropped to include only the central portion of the rotated image and is the same size as A.

Image Transformation (Rotation)



a b c d

FIGURE 2.36 (a) A 300 dpi image of the letter T. (b) Image rotated 21° clockwise using nearest neighbor interpolation to assign intensity values to the spatially transformed pixels. (c) Image rotated 21° using bilinear interpolation. (d) Image rotated 21° using bicubic interpolation. The enlarged sections show edge detail for the three interpolation approaches.